Towers of Hanoi (3 Pegs, 3 Disks): Problem Formulation and Solution Report

Prepared by: D. Jeshurun Silas

Registration Number: 192525324

Date: October 20, 2026

Subject: Artificial Intelligence

1. Aim

To formulate the Pegs and Disks problem (Towers of Hanoi) with 3 pegs and 3 disks as a state-space search problem, and to find the optimal sequence of moves that reaches the goal state.

2. Problem Statement

There are three pegs (A, B and C) and three disks of different sizes: red (largest), blue (medium) and green (smallest). A move takes the topmost disk of any peg and places it on top of any other peg. A larger disk may never be placed on a smaller disk. In the goal state, all disks are stacked on peg B, with green on top, blue in the middle and red at the bottom, as shown in the figure.

The problem statement does not specify the initial state, so the standard configuration is assumed: all three disks stacked on peg A in the same order.

3. Problem Formulation

Component	Description
State	The peg holding each disk, written as (Red, Blue, Green). For example, (A, A, A) means all disks are on peg A.
Initial state	(A, A, A): all disks on peg A
Goal state	(B, B, B): all disks on peg B
Operators	Move the topmost disk of one peg to another peg, provided the target peg is empty or its top disk is larger

Component	Description
Path cost	1 per move
Number of states	3 × 3 × 3 = 27 (each disk can be on one of 3 pegs; the stacking order on a peg is forced)
Branching factor	At most 3 legal moves per state (exactly 2 in the initial state)

4. Optimal Solution

The minimum number of moves for n disks is $2^n - 1$. For n = 3 this gives 7 moves. The table shows each move and the resulting configuration, with disks listed from bottom to top on each peg.

Step	Move	Peg A	Peg B	Peg C	State (R, B, G)
0	Start	Red, Blue, Green	empty	empty	(A, A, A)
1	Green: A → B	Red, Blue	Green	empty	(A, A, B)
2	Blue: A → C	Red	Green	Blue	(A, C, B)
3	Green: B → C	Red	empty	Blue, Green	(A, C, C)
4	Red: A → B	empty	Red	Blue, Green	(B, C, C)
5	Green: C → A	Green	Red	Blue	(B, C, A)
6	Blue: C → B	Green	Red, Blue	empty	(B, B, A)
7	Green: A → B	empty	Red, Blue, Green	empty	(B, B, B)

5. Search Analysis

Because every move has the same cost, Breadth-First Search (BFS) finds an optimal solution. It expands states level by level, so the first time it reaches (B, B, B) it has found a path with the fewest moves, which is 7.

Property	Value for this problem
Total reachable states	27

Property	Value for this problem
Shortest solution length	7 moves
Complete	Yes, since the state space is finite
Optimal (BFS)	Yes, since all step costs are equal
Repeated states	Possible (moves can be undone), so a visited-state check is needed to avoid loops

The recursive idea behind the 7-move solution is: move the top $n - 1$ disks to the spare peg, move the largest disk to the target peg, then move the $n - 1$ disks onto it. This gives the recurrence $T(n) = 2T(n - 1) + 1$, which solves to $T(n) = 2^n - 1$.

6. Conclusion

The 3-peg, 3-disk problem was formulated with a compact state representation, a clear goal test and simple unit-cost operators. The state space has 27 states, and the optimal solution from (A, A, A) to (B, B, B) takes 7 moves, matching the $2^n - 1$ formula. The problem is a standard example of how recursion and uninformed search such as BFS solve well-defined puzzles.